



University of Asia Pacific
Department of Computer Science and Engineering
CSE 450: VLSI Lab

LAB REPORT

Experiment Number: 03

Experiment Title: Design and Simulation of an 8-Bit Full Adder Using NAND-Based Structural VHDL in Xilinx ISE

Submitted by:

Name: Rifah Tasfia

Student ID: 22201126

Section: C1

Submitted to:

Md. Rasheduzzaman

Lecturer

Department of Computer Science and Engineering

Date of Submission: 30/09/2026

1. Experiment Name

Design and Simulation of an 8-Bit Full Adder Using NAND-Based Structural VHDL in Xilinx ISE

2. Objective

In this experiment, we want to:

1. Learn how basic digital logic gates are built and how they work.
2. Write a NAND gate in VHDL.
3. Build other gates by using copies (instances) of the NAND gate.
4. Build an XOR gate from NAND gates.
5. Build a Half Adder from an XOR gate and an AND gate.
6. Build a Full Adder from two Half Adders and one OR gate.
7. Learn structural modeling in VHDL.
8. Simulate the circuits in Xilinx ISE.
9. Check the simulation results against the truth tables.
10. Learn hierarchical design, where big circuits are made from small parts that can be used again.
11. Build and simulate an 8-bit Full Adder by joining eight 1-bit Full Adders in structural VHDL.

3. Apparatus / Hardware & Software Requirements

- List all required tools and components:

- Xilinx ISE 14.7
- VHDL
- ISim Simulator
- A computer for design and simulation

4. Introduction

A Full Adder is a basic digital circuit that adds three binary inputs. Its inputs are:

- **A:** the first input bit
- **B:** the second input bit
- **Cin:** the carry coming in

It gives two outputs:

- **Sum:** the result of the addition
- **Cout:** the carry going out

The formulas for a Full Adder are:

$$\text{Sum} = A \oplus B \oplus \text{Cin}$$

$$\text{Cout} = A \cdot B + (A \oplus B) \cdot \text{Cin}$$

We built the Full Adder step by step using a hierarchy. First, we made a NAND gate. Then we used NAND gates to make the other basic gates. Next, we made a Half Adder from XOR and AND gates. Last, we made a Full Adder from two Half Adders and an OR gate.

The order of the design is:

NAND Gate → Basic Logic Gates → Half Adder → Full Adder

5. NAND Gate Design

A NAND gate is a universal logic gate. A two-input NAND gate gives the opposite result of an AND operation.

The Boolean expression is:

$$Y = \overline{A \cdot B}$$

Truth Table:

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

The NAND gate was implemented using VHDL Dataflow modeling.

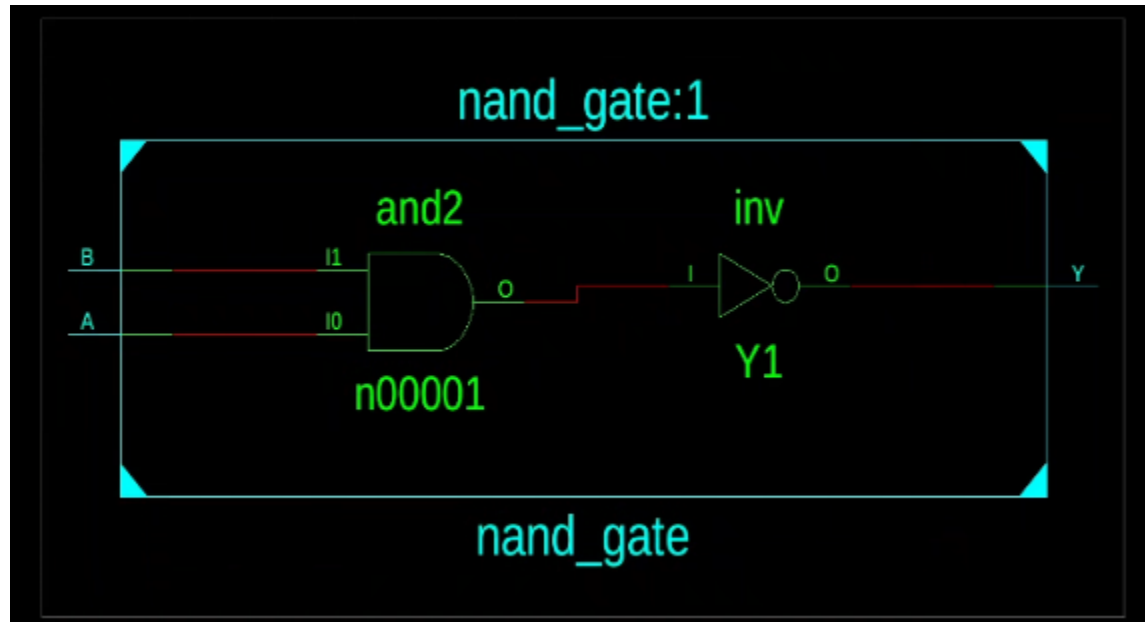


Figure 1: RTL schematic of the NAND gate.

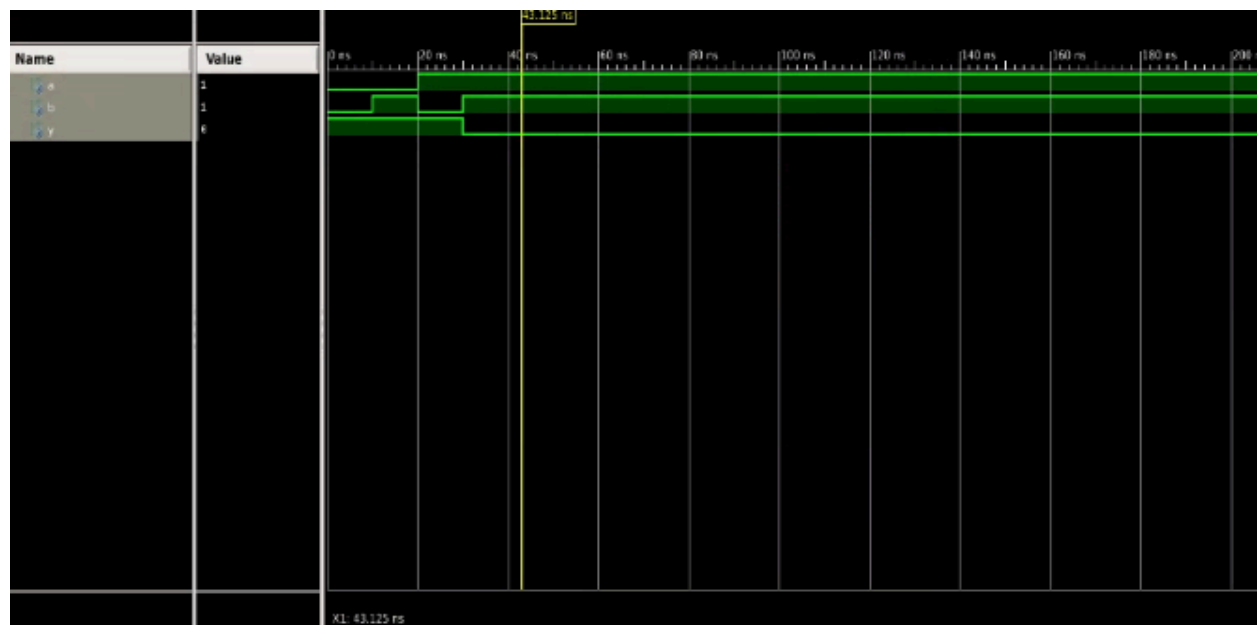


Figure 2: Simulation waveform of the NAND gate.

6. Construction of Basic Gates Using NAND Gates

NAND is a universal gate, so other logic gates can also be made by using NAND gates.

The following gates were designed using NAND gate components:

- AND
- OR
- NOR
- XOR

These gates were implemented using structural VHDL modeling.

6.1 AND Gate Using NAND Gates

An AND gate can be made using two NAND gates.

First: $N1 = A \text{ NAND } B$

Then: $Y = N1 \text{ NAND } N1$

Therefore: $Y = A \cdot B$

Number of NAND gates: 2 NAND gates

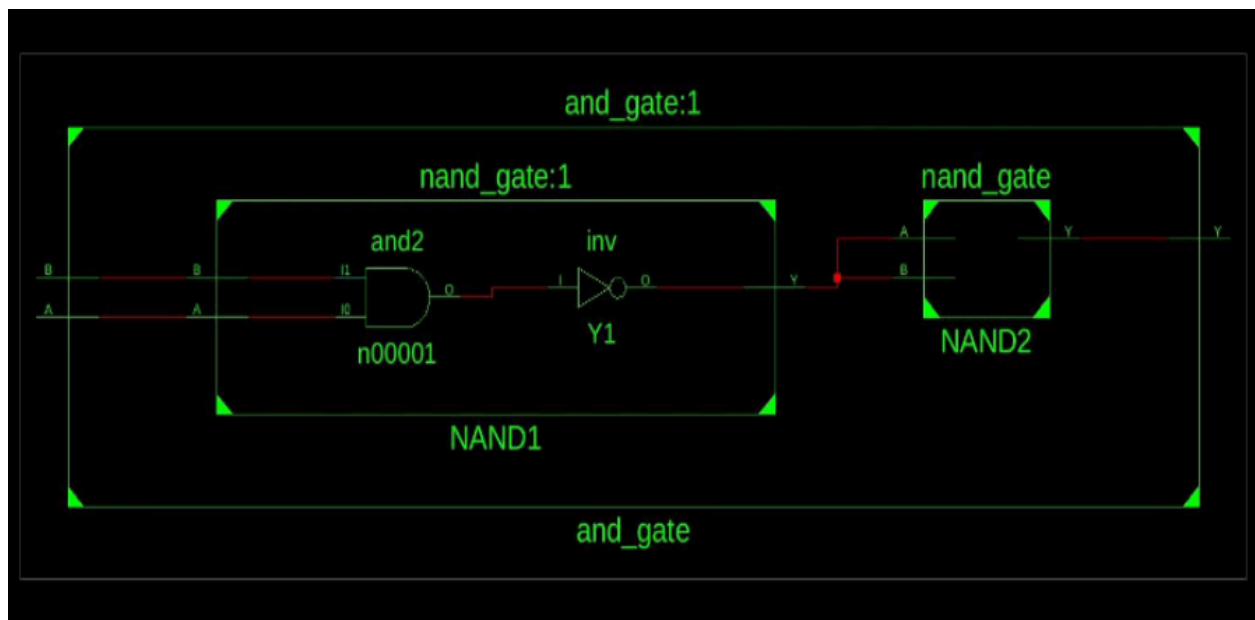


Figure 3: RTL schematic of the NAND-based AND gate.

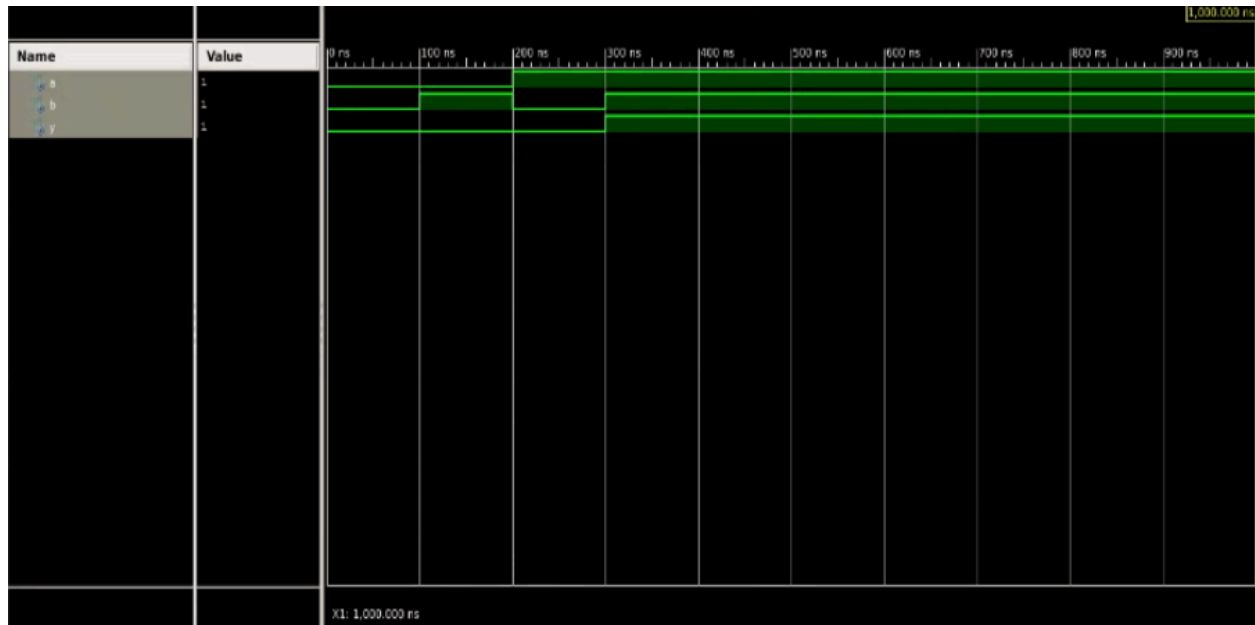


Figure 4: Simulation waveform of the AND gate.

6.2 OR Gate Using NAND Gates

Using De Morgan's theorem:

$$A + B = (\bar{A} \cdot \bar{B})$$

Therefore:

$$N1 = A \text{ NAND } A$$

$$N2 = B \text{ NAND } B$$

$$Y = N1 \text{ NAND } N2$$

So, an OR gate needs 3 NAND gates.

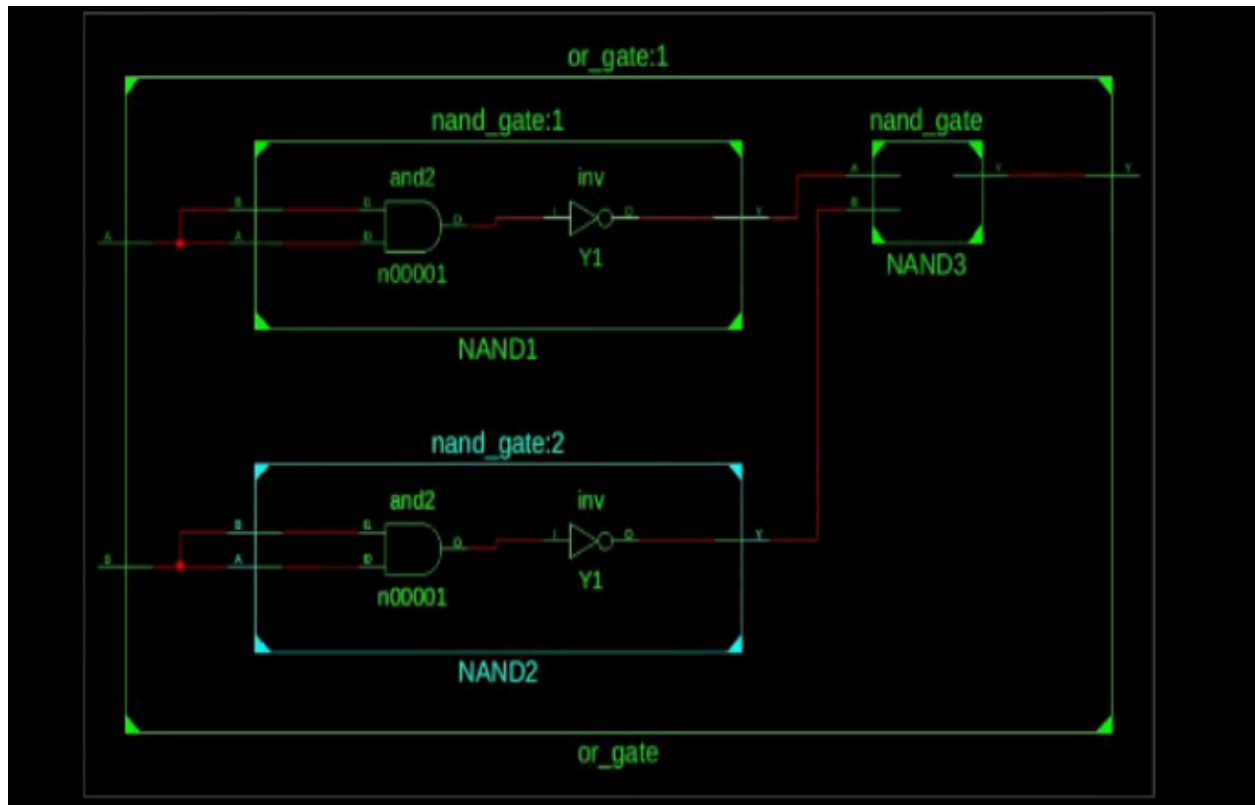


Figure 5: RTL schematic of the NAND-based OR gate.

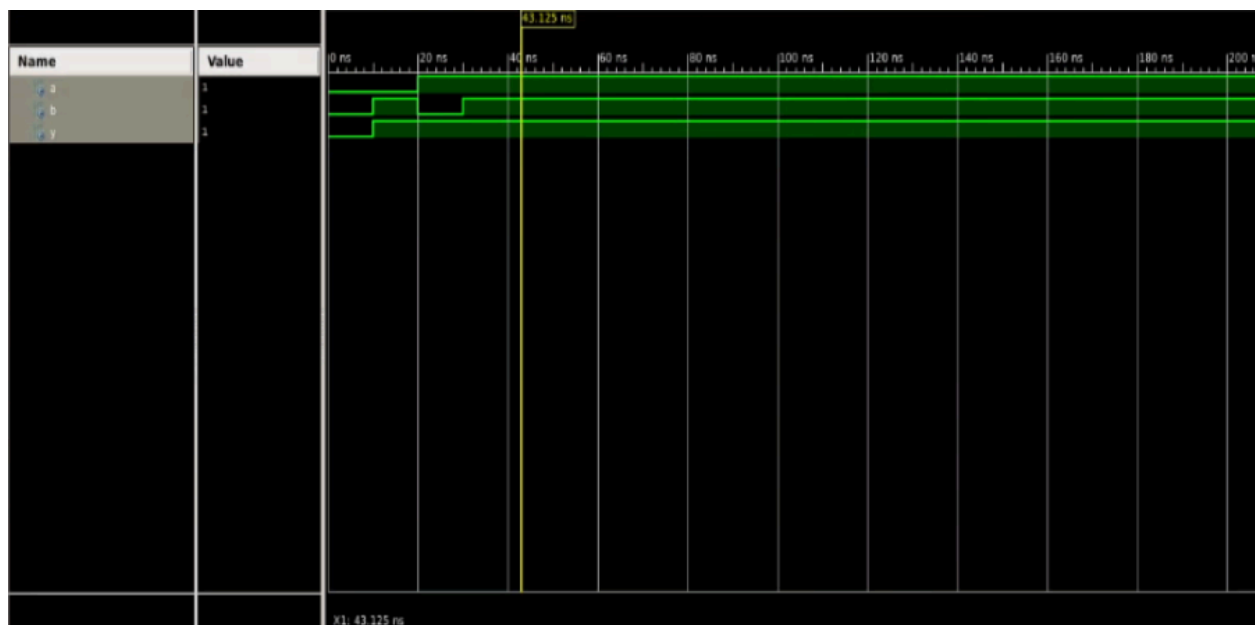


Figure 6: Simulation waveform of the OR gate.

6.3 NOR Gate Using NAND Gates

A NOR gate can be made by first creating an OR operation and then inverting its output.

This design uses four NAND gates:

$N1 = A \text{ NAND } A$

$N2 = B \text{ NAND } B$

$N3 = N1 \text{ NAND } N2$

$Y = N3 \text{ NAND } N3$

Therefore: $Y = (A + B)^{\overline{}}$

Number of NAND gates: 4 NAND gates

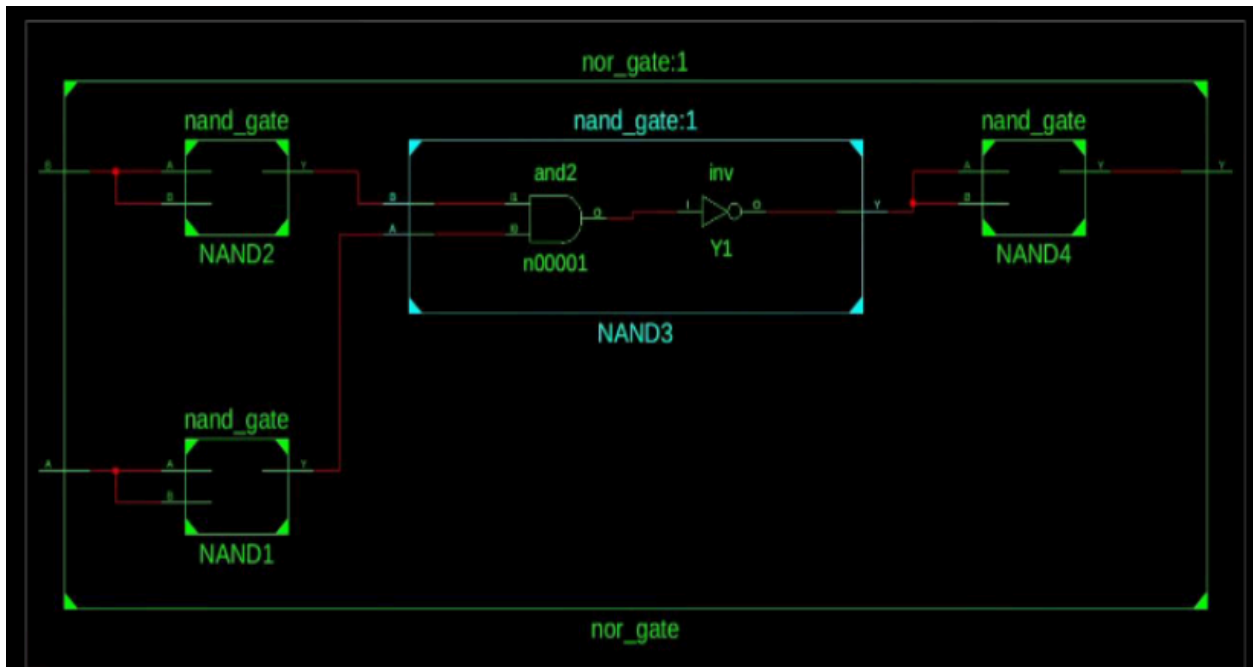


Figure 7: RTL schematic of the NAND-based NOR gate.

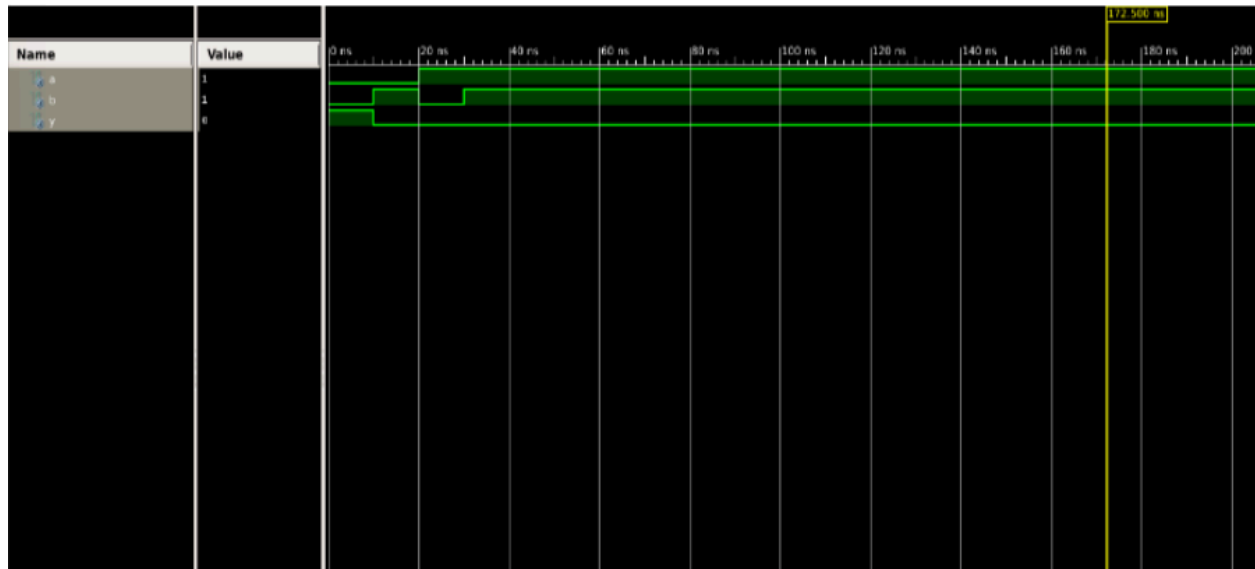


Figure 8: Simulation waveform of the NOR gate.

6.4 XOR Gate Using NAND Gates

An XOR gate gives logic 1 when its two inputs are different.

The NAND-only design uses four NAND gates:

$$N1 = A \text{ NAND } B$$

$$N2 = A \text{ NAND } N1$$

$$N3 = B \text{ NAND } N1$$

$$Y = N2 \text{ NAND } N3$$

Therefore: $Y = A \oplus B$

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

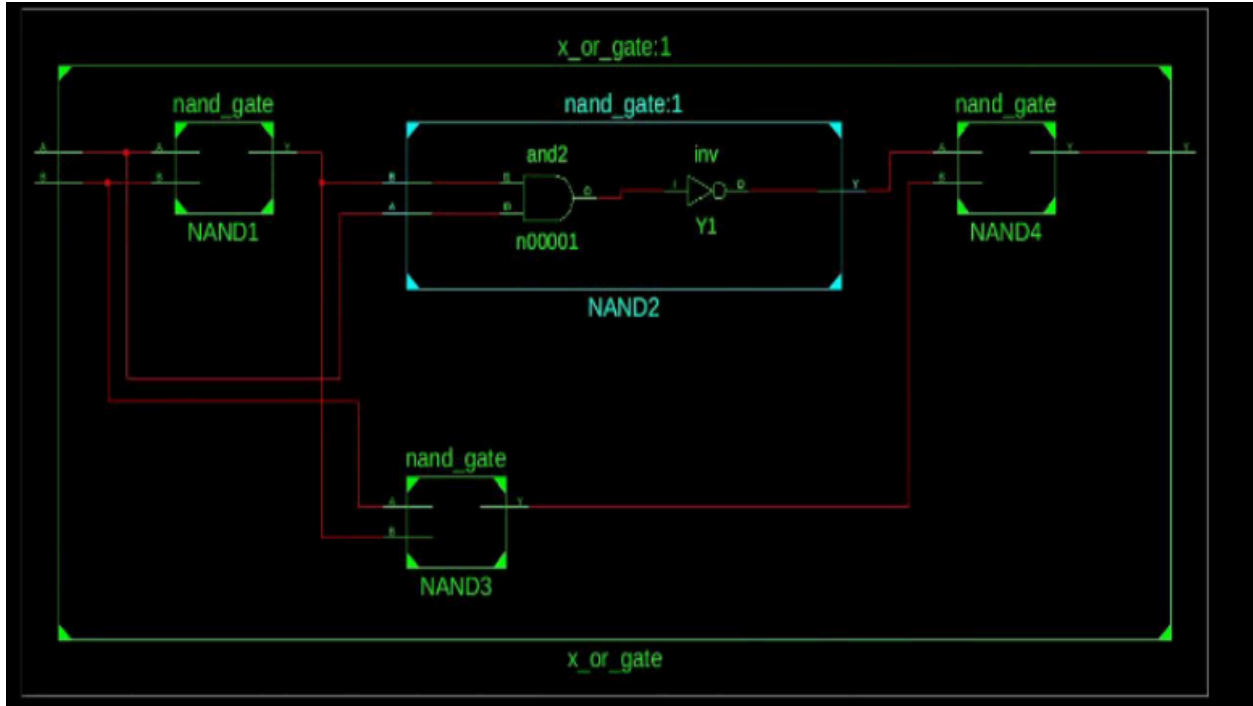


Figure 9: RTL schematic of the NAND-based XOR gate.

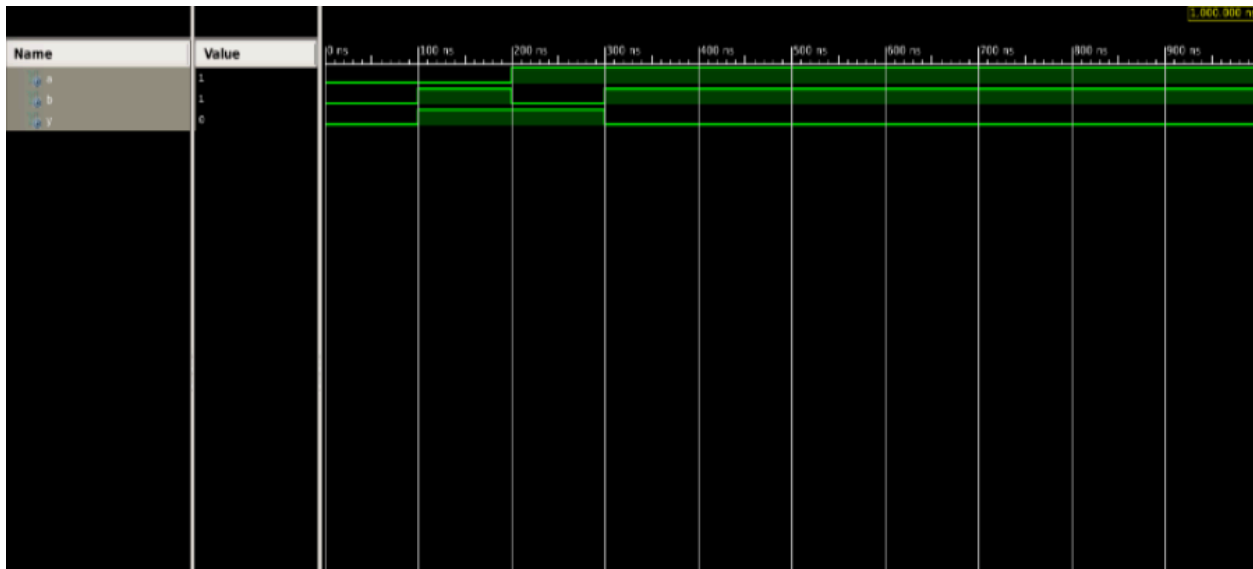


Figure 10: Simulation waveform of the XOR gate.

7. Half Adder Design

A Half Adder is a combinational circuit that adds two one-bit binary numbers.

It has two inputs:

- A
- B

It has two outputs:

- Sum
- Carry

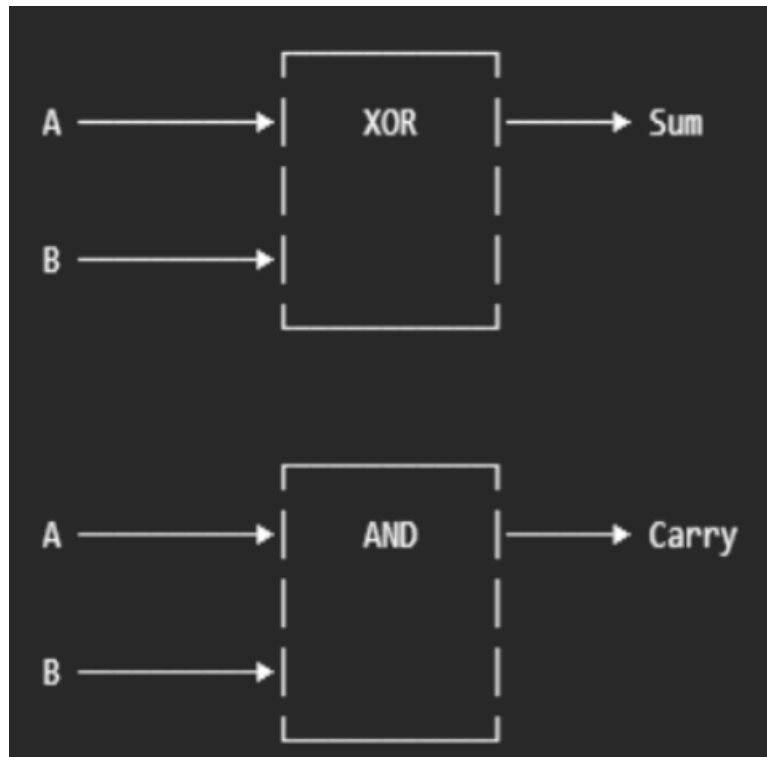
The Boolean expressions are:

$$\text{Sum} = A \oplus B$$

$$\text{Carry} = A \cdot B$$

The Half Adder was designed using the XOR and AND components created earlier.

Block Diagram



Half Adder Truth Table

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

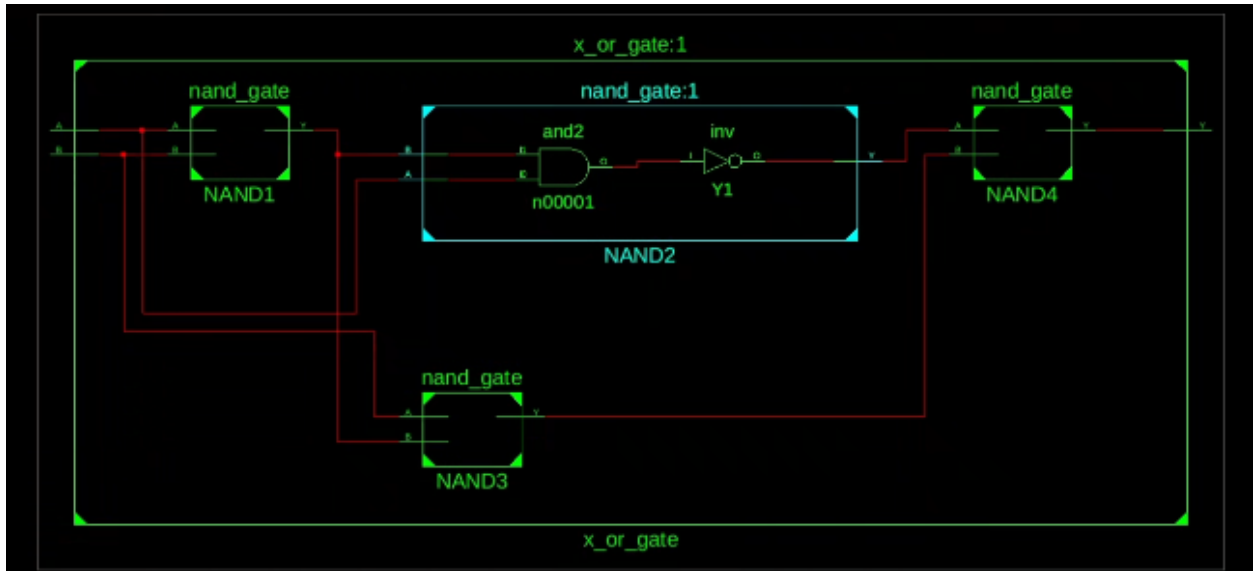


Figure 11: RTL schematic of the Half Adder.

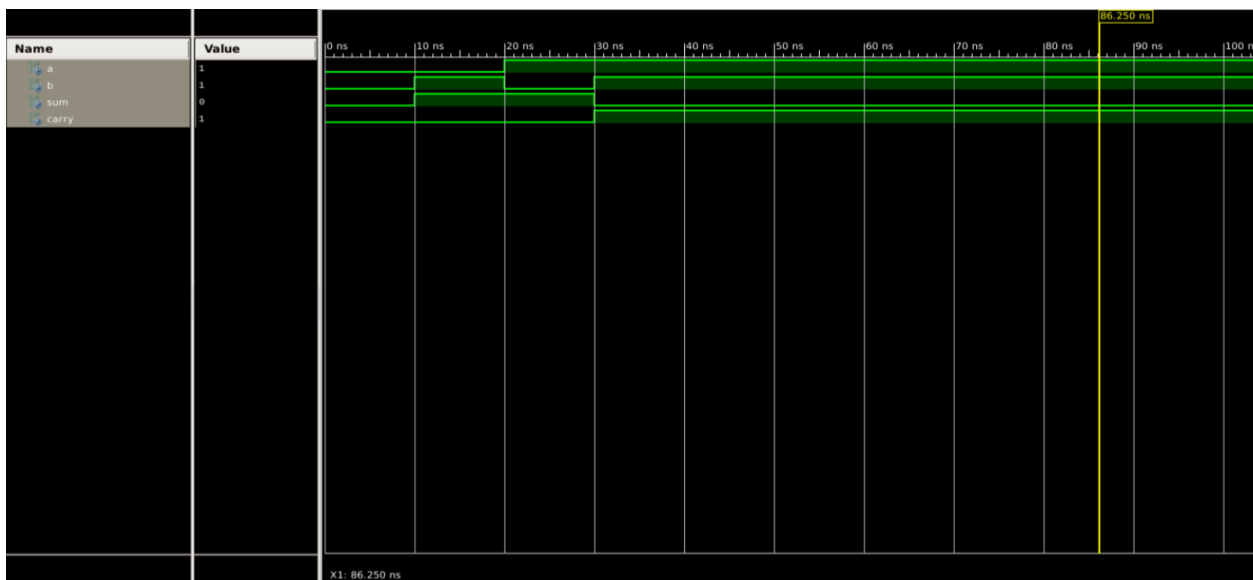


Figure 12: Simulation waveform of the Half Adder.

8. Full Adder Design

A Full Adder adds three one-bit binary values.

Inputs:

- A
- B
- Cin

Outputs:

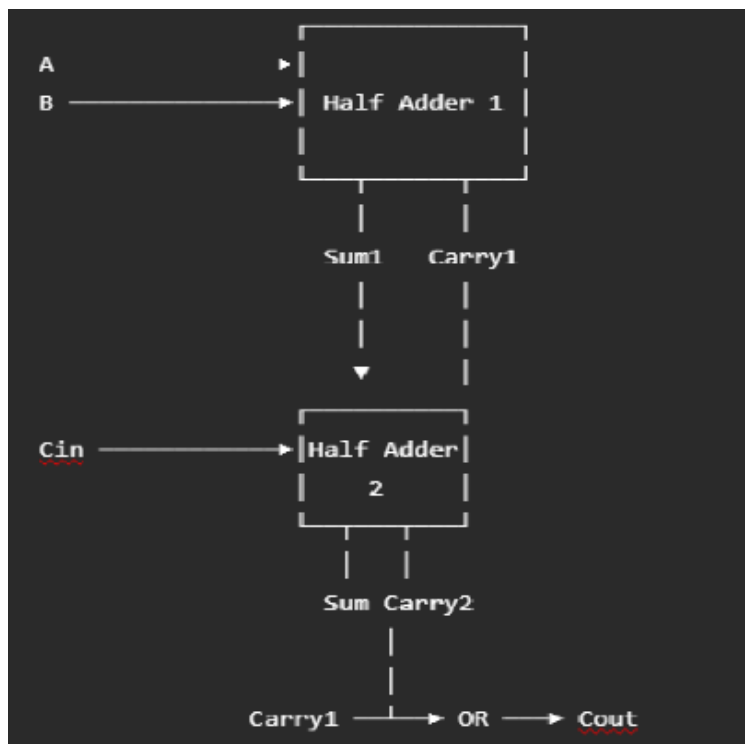
- Sum
- Cout

A Full Adder can be built using:

- Two Half Adders
- One OR gate

The first Half Adder adds A and B. The second Half Adder adds the first Half Adder's Sum with Cin. Finally, the two carry outputs are combined using an OR gate.

8.1 Full Adder Block Diagram



The Full Adder is arranged using two Half Adders and one OR gate.

9. Full Adder Logic

The first Half Adder produces:

$$\text{Sum1} = A \oplus B$$

$$\text{Carry1} = A \cdot B$$

The second Half Adder produces:

$$\text{Sum} = \text{Sum1} \oplus \text{Cin}$$

$$\text{Carry2} = \text{Sum1} \cdot \text{Cin}$$

Finally:

$$\text{Cout} = \text{Carry1} + \text{Carry2}$$

Therefore:

$$\text{Sum} = A \oplus B \oplus \text{Cin}$$

$$\text{Cout} = AB + (A \oplus B)\text{Cin}$$

10. Full Adder Truth Table

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

The simulation results should match this truth table for all eight possible input combinations.

11. RTL Schematic of the Full Adder

The final RTL schematic should show the hierarchical structure of the Full Adder.

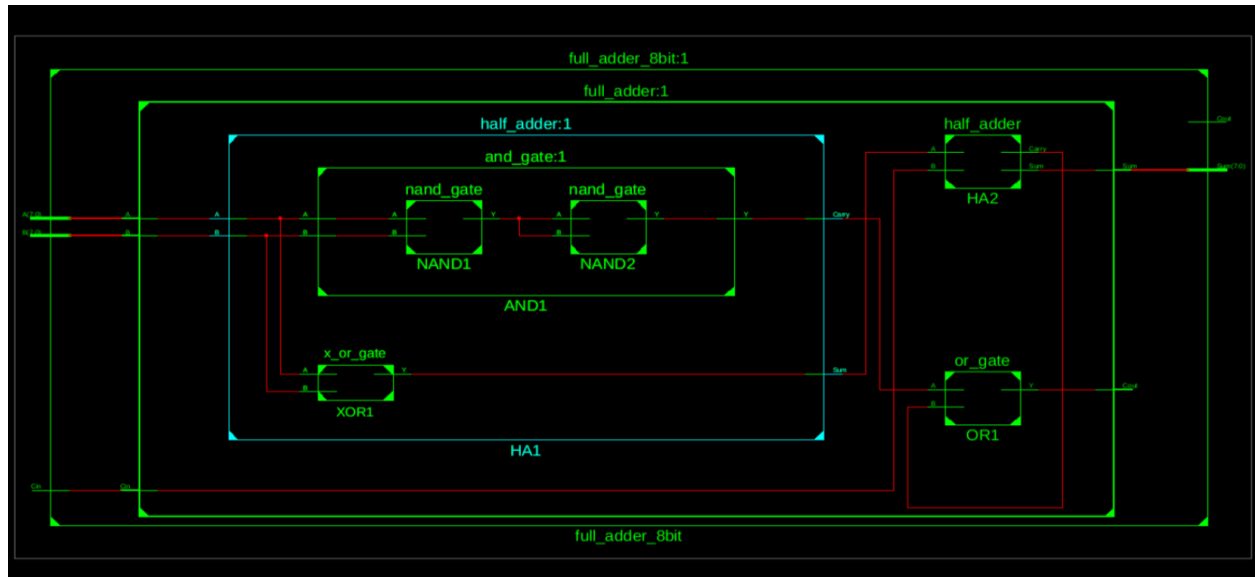


Figure 13: RTL schematic of the complete Full Adder.

This is an important screenshot in the report, so the RTL schematic should be clearly visible.

12. Simulation of the 8-Bit Full Adder

We simulated the whole 8-bit Full Adder with the ISim simulator in Xilinx ISE. The testbench gives the circuit different pairs of 8-bit inputs, A and B, along with the carry input Cin. We then read the 8-bit Sum and the final Cout from the waveform.

This adder is a combinational circuit, so it does not need a clock. We simply change the inputs after fixed time gaps and check that the circuit works correctly.

Test cases used:

A	B	Cin	Expected Sum	Cout
00000000	00000000	0	00000000	0
00000001	00000001	0	00000010	0
00000101	00000011	0	00001000	0
00001111	00001010	0	00011001	0
01100100	00110010	0	10010110	0
11111111	00000001	0	00000000	1
11111111	11111111	0	11111110	1
11111111	11111111	1	11111111	1

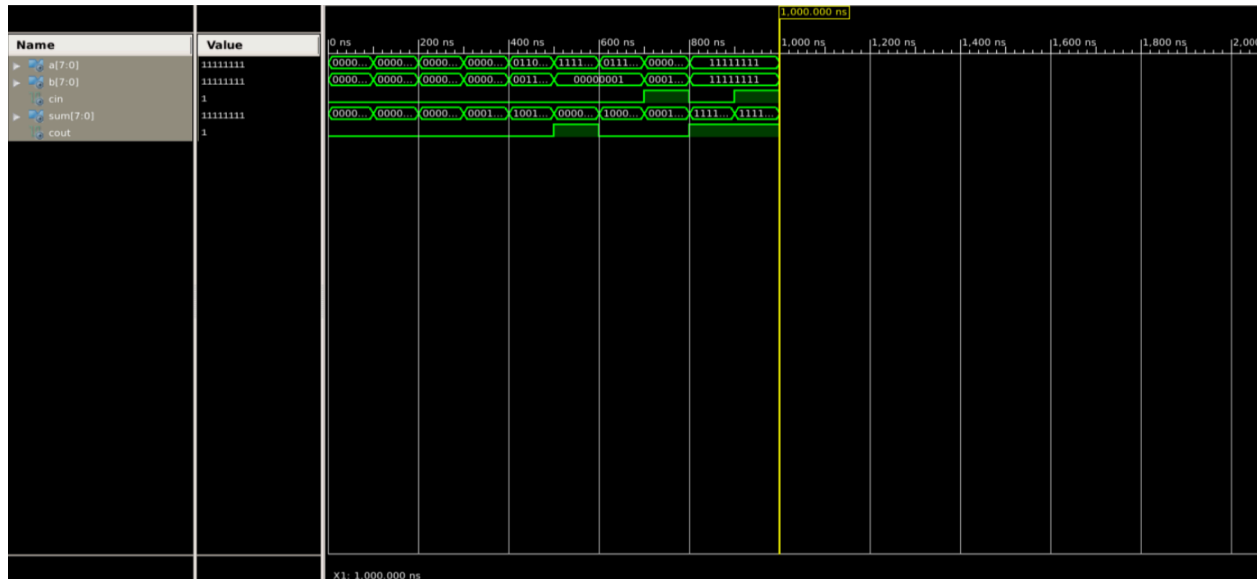


Figure 14: Simulation waveform showing all eight input combinations and their corresponding Sum and Cout outputs.

The waveform shows A[7:0], B[7:0], Cin, Sum[7:0], and Cout, zoomed in so each of the eight test cases can be seen clearly.

[Insert Figure 14: Simulation waveform showing the eight test cases and their Sum and Cout outputs]

It is better to zoom the waveform so that all eight input combinations can be seen clearly.

13. 8-Bit Full Adder Simulation Result Analysis

The simulation confirms that the designed Full Adder works correctly.

Case 1

- A = 00000001, B = 00000001, Cin = 0
- Sum = 00000010, Cout = 0
- Meaning: $1 + 1 = 2$

Case 2

- A = 11111111, B = 00000001, Cin = 0
- Sum = 00000000, Cout = 1
- Meaning: $255 + 1 = 256$

Case 3

- $A = 11111111$, $B = 11111111$, $C_{in} = 1$
- $Sum = 11111111$, $C_{out} = 1$
- Meaning: $255 + 255 + 1 = 511$

The simulated results match the expected math results, so the 8-bit Full Adder works correctly.

14. GitHub Repository

All VHDL source codes, testbenches, and related project files should be uploaded to a GitHub repository.

<https://github.com/Rifah03/VLSI-ISE-14.7>

15. Conclusion

In this experiment, we designed and simulated an 8-bit Full Adder using VHDL and Xilinx ISE. We used a hierarchical, structural method. We first made a NAND gate and then used it to build the other basic gates.

We used the XOR and AND gates to make a Half Adder. Then we used two Half Adders and an OR gate to make a 1-bit Full Adder. Finally, we joined eight 1-bit Full Adders in cascade to make the complete 8-bit Full Adder.

The circuit takes two 8-bit binary inputs, A and B, and an starting carry input, C_{in} . It gives an 8-bit Sum and a final carry output, C_{out} . The carry output of each Full Adder goes into the carry input of the next one.

We simulated the design with ISim in Xilinx ISE. We tried different inputs, and the Sum and C_{out} values matched the expected results. So the 8-bit Full Adder was designed, built, and tested successfully.

This experiment shows how structural VHDL, component instantiation, hierarchical design, and reusable building blocks can be used to make a bigger combinational circuit.